Contents

Contents **i**

List of Figures **ii**

List of Tables **iii**

I Introduction **1**

II What is TAMPINES **2**

IIISteam Tables **3**

IVRankine Cycle Derivations **4**

1 Mass, Energy and Momentum Balance **6**

1.1 Raw forms of mass, momentum and energy balance 6
1.2 converting forms to mass flowrate 6
1.3 Approach to Solving nonlinear Equations 7
1.4 PISO Algorithm 11
1.5 SIMPLE Algorithm 12
1.6 PIMPLE Algorithm 14
1.7 Application to Rankine Cycle 15

Bibliography **31**

List of Figures

List of Tables

Acknowledgments

Part I

Introduction

Part II

What is TAMPINES

Part III

Steam Tables

Part IV

Rankine Cycle Derivations

In this part, we consider a four control volume system representing a simple Rankine cycle. This will form the theoretical basis to

Chapter 1

Mass, Energy and Momentum Balance

In OpenFOAM, the PIMPLE algorithm is often used to solve for incompressible fluid flow [Devolder et al., 2015], this is one of the many standard solvers for solving flow in OpenFOAM [Penttinen, Yasari, and Nilsson, 2011]. Jasak has published this in his PhD thesis, available online, [Jasak, 1996] on page 143-149.

1.1 Raw forms of mass, momentum and energy balance

Let's consider mass balance and momentum balance:

$$\frac{\partial m}{\partial t} = \sum_i m_i \tag{1.1}$$

$$\frac{\partial(\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u}\mathbf{u}) = -\nabla P + \nabla \cdot \tau + \rho g \tag{1.2}$$

$\nabla \cdot \tau$ represents shear, ρg represents the gravity body force, $-\nabla P$ represents the pressure gradient.

$$V_j \frac{\partial \rho_i h_i}{\partial t} + \sum_j \dot{m}_j h_j = Q_i - W_i + \sum_j \tau_j \cdot \mathbf{u}_j \tag{1.3}$$

Where $+\sum_j \tau_j \cdot \mathbf{u}_j$ is basically the friction loss terms where energy is transformed from kinetic energy to sensible heat. Q and W represent the thermodynamic work

1.2 converting forms to mass flowrate

It is often more convenient to convert the momentum equations to be in terms of mass flowrates. This is for 1D case.

To do so, we transform this by consider this 3D equation.

$$\frac{\partial(\rho\mathbf{u})}{\partial t} + \nabla \cdot (\rho\mathbf{u}\mathbf{u}) = -\nabla P + \nabla \cdot \tau + \rho g \tag{1.4}$$

We need to consider just the x component, for a simple 1D case.

Then multiply by the cross sectional area.

1.3 Approach to Solving nonlinear Equations

Jasak in his PhD thesis outlines the methodology for solving this set of equation [Jasak, 1996] on page 143-149. These were the foundations for what would eventually become OpenFOAM Jasak, Jemcov, Tukovic, et al., 2007 licensed under GPLv3.

The momentum equation is quadratic in u, which makes the equation solution nonlinear. Solving this nonlinear equation would either need a nonlinear solver such as Newton Raphson method, or solving a linearised form of these equations, and iterating to convergence. The latter was chosen [Jasak, 1996].

$$\nabla \bullet (\mathbf{u}\mathbf{u}) = \sum_f \mathbf{S} \cdot \mathbf{u}_f \mathbf{u}_f = \sum_f F\mathbf{u}_f = a_p\mathbf{u}_p + \sum_N a_N\mathbf{u}_n \tag{1.5}$$

Finite Volume Discretisaion, Face and Centroid Properties

Section 3.3.1 within Jasak's thesis explains these terms.

First, we need to understand that Gauss's theorem is used in Finite Volume analysis, which is the bedrock of OpenFOAM solvers. $\sum_f \mathbf{S} \cdot \mathbf{u}_f \mathbf{u}_f$ represent the surface vector $\mathbf{S}$ and this will have a dot product with the velocity at the faces of the control volume $\mathbf{u}_f$.

Note that the subscript f represents values at the faces.

$\sum_f \mathbf{S} \cdot \mathbf{u}_f \mathbf{u}_f$ can be written in shorthand $\sum_f F\mathbf{u}_f$.

Now, we may not have the values of the velocities or fluxes at the faces. So we have to estimate it based on the properties at the centroid of the cell denoted p, and those of the neighbours denoted n.

Thus, the surface values denoted f are estimated based on the values at the cell centroids p and n based on some kind of interpolation scheme. This can be linear interpolation, or some upwind differencing scheme. This is described in page 81 of Jasak's thesis [Jasak, 1996].

With linear interpolation at the face, the solution can become unbounded. With upwind differencing, the solution is bounded, but diffusive. Much of the discussion revolves around how to estimate properties at the faces. This is why a whole host of differencing schemes comes up. Eg. at page 110 of Jasak's thesis [Jasak, 1996].

Linearisation and Predictor values

Given a very nonlinear system of equations. It was desirable to consider solving a linearised version of the momentum equations. We want to solve for the velocity $\mathbf{u}^{t+\Delta t}$ at the next timestep. But we only know $\mathbf{u}^t$.

We know that the face fluxes should be based on values in the cell centroids for both the cell p and that of the neighbouring cell n, hence we use:

$$a_p \mathbf{u}_p + \sum_N a_N \mathbf{u}_n$$

However, we have $\mathbf{uu}$ terms. To solve this problem and linearise this, we can use $\mathbf{u}^t \mathbf{u}^{t+\Delta t}$. Basically, $\mathbf{u}^t$ is known. This is the velocity at the previous timestep, and is therefore a constant. This will then help form the respective a_p and a_n constants so that the equations are linear.

Let's go back to our equations:

$$\frac{\partial(\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{uu}) = -\nabla P + \nabla \cdot \tau + \rho g \tag{1.6}$$

In the incompressible assumption,

$$\frac{\partial(\mathbf{u})}{\partial t} + \nabla \cdot (\mathbf{uu}) = -\nabla P/\rho + \nabla \cdot \tau/\rho + g \tag{1.7}$$

We can do time discretisation:

$$\frac{\mathbf{u}^{t+\Delta t} - \mathbf{u}^t}{\Delta t} + \nabla \cdot (\mathbf{uu}) = -\nabla P/\rho + \nabla \cdot \tau/\rho + g \tag{1.8}$$

Now, remember, based on divergence theorem we need fluxes or properties at the faces:

$$\frac{\mathbf{u}^{t+\Delta t} - \mathbf{u}^t}{\Delta t} + \sum_f \mathbf{S} \cdot \mathbf{u}_f \mathbf{u}_f = -\nabla P/\rho + \nabla \cdot \tau/\rho + g \tag{1.9}$$

And since we cannot find properties at the faces, we have to estimate them using properties at the centroids.

Just to be clear, let's label the velocities at their proper locations, and then linearise the equation using some semi implicit time discretisation:

$$\frac{\mathbf{u}_p^{t+\Delta t} - \mathbf{u}_p^t}{\Delta t} + \sum_f \mathbf{S} \cdot \mathbf{u}_f^t \mathbf{u}_f^{t+\Delta t} = -\nabla P/\rho + \nabla \cdot \tau/\rho + g \tag{1.10}$$

The fluxes at the faces can be further estimated using some interpolation scheme. The easiest to understand is a linear scheme. Herein, we use some weighting factor to estimate the flux at the face between the cell and the other cell.

$$\mathbf{u}_f^t = w\mathbf{u}_p^t + (1-w)\mathbf{u}_n^t$$

Substituting this back:

$$\frac{\mathbf{u}_p^{t+\Delta t} - \mathbf{u}_p^t}{\Delta t} + \sum_f \mathbf{S}\cdot(w\mathbf{u}_p^t + (1-w)\mathbf{u}_n^t)(w\mathbf{u}_p^{t+\Delta t} + (1-w)\mathbf{u}_n^{t+\Delta t}) = -\nabla P/\rho + \nabla\cdot\tau/\rho + g \quad (1.11)$$

To make life easier for us, we can choose not to even bother with explicitly solving for the velocities of the neigbours

$$\frac{\mathbf{u}_p^{t+\Delta t} - \mathbf{u}_p^t}{\Delta t} + \sum_f \mathbf{S}\cdot(w\mathbf{u}_p^t + (1-w)\mathbf{u}_n^t)(w\mathbf{u}_p^{t+\Delta t} + (1-w)\mathbf{u}_n^t) = -\nabla P/\rho + \nabla\cdot\tau/\rho + g \quad (1.12)$$

Messy as the equation is, we now have a linear equation in velocities for each centroid at the next timestep $u_p^{t+\Delta t}$ and $u_n^{t+\Delta t}$.

We can then simplify this equation as such:

$$a_p\mathbf{u}_p^{t+\Delta t} = \mathbf{H}(\mathbf{u}^t) - \nabla P/\rho + \nabla\cdot\tau/\rho + g \quad (1.13)$$

$$\mathbf{u}_p^{t+\Delta t} = \frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p} + \frac{\nabla\cdot\tau/\rho}{a_p} + g/a_p \quad (1.14)$$

Of course, the estimates for $\mathbf{u}^{t+\Delta t}$ are not going to be accurate. They are initial guess or **predicted** values.

We need to do some further correction to these predicted values in order to arrive at the correct answer.

Corrector steps

So far, we've only bothered with the momentum equation and ignored mass balance (or continuity in this case). Thus, there is no guarantee that the mass balance is fulfilled.

The mass balance is:

$$\nabla\cdot\mathbf{u}_p = 0$$

We then apply Gauss's divergence theorem to obtain:

$$\nabla\cdot\mathbf{u}_p = \sum_f \mathbf{S}\cdot\mathbf{u}_f = 0$$

Now again, how do we deal with the face fluxes again? We need to interpolate.

Suppose again that we can use some weighting factor (linear interpolation scheme), that on a face between a point and its neighbouring cell:

$$u_f = wu_p + (1-w)u_n$$

$$\mathbf{u}_p^{t+\Delta t} = \frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p$$

since

$$\mathbf{u}_p^{t+\Delta t} = 0$$

Then:

$$\nabla \cdot \left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p \right) = 0$$

Moving the pressure term to the right:

$$\nabla \cdot \left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p \right) = \nabla \cdot \frac{P/\rho}{a_p}$$

This is equation 3.141 in page 146 of Jasak's thesis [Jasak, 1996]. Of course, τ and g were not included.

Now, if we apply divergence theorem on the H terms:

$$\nabla \cdot \left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p} \right) = \sum_f \mathbf{S}_f \cdot \left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p} \right)_f$$

We also need to discretise the pressure side

$$\nabla \cdot \frac{P/\rho}{a_p} = \sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p} \right)_f$$

With this discretisation, we obtain

$$\sum_f \mathbf{S}_f \cdot \left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p} \right)_f = \sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p} \right)_f$$

With the interpolation:

$$u_f = wu_p + (1-w)u_n$$

the matrices can be set up. This is equation 3.143 in Jasak's thesis [Jasak, 1996].

So basically, the continuity equation is combined with the momentum equation to give a combined corrector equation.

1.4 PISO Algorithm

Pressure Implicit Splitting of Operators (PISO).

This includes a few steps, outlined in Jasak's thesis [Jasak, 1996]:

Step 1

This is the momentum predictor step, we can predict a velocity field using the previous steps

$$\mathbf{u}_p^{predictor} = \frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p \tag{1.15}$$

Step 2

With this, we can formulate a new $\frac{\mathbf{H}(\mathbf{u}^t)}{a_p}$:

$$\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p} \tag{1.16}$$

With this, we can substitute into the mass balance (pressure equation):

$$\sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p}\right)_f = \sum_f \mathbf{S}_f \cdot \left(\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p}\right)_f$$

This allows for a new pressure field. Note that the gravity terms are missing because hydrostatic pressure is ignored.

$$\sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p}\right)_f$$

It is conceivable that this results in some small matrix where pressure fields are simultaneously solved.

This is the pressure solution within Jasak's thesis page 147 [Jasak, 1996].

Step 3

We now have a new pressure field based on mass balance:

$$\sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p}\right)_f$$

And a new predicted H field based on the initial predictor step:

$$\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p}$$

Now, the predictor H field is not guaranteed to allow continuity with the old velocity, but since it was used to generate a new pressure field using the continuity equation, it should be able to guarantee creation of a new velocity field which will satisfy continuity equation.

$$\mathbf{u}_p^{corrected} = \frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p} - \left(\frac{\nabla P/\rho}{a_p}\right)_{\text{corrected pressure field}} \tag{1.17}$$

repeated iterations

This forms the basis for iterating through both mass and momentum balances. This is one round of PISO prediction-correction.

We may iterate this several times until the residual is small between consecutive iterations.

Now, it must be noted that during PISO, we do NOT repeat step 1 to recalculate as mentioned in page 148 Jasak's thesis Jasak, 1996:

$$\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p}$$

The H term is kept constant through the whole correction sequence and changed in only the next momentum predictor [Jasak, 1996].

1.5 SIMPLE Algorithm

The SIMPLE algorithm is similar, except that it is solving for steady state solutions, and uses large time step.

The key difference is rather than skip Step 1 within loops, we repeat step 1 for momentum predictor. This is the key difference between PISO and SIMPLE. It is that we run the momentum predictor step every time in SIMPLE. In this regard, it is more important to re-calculate H in every iteration so that non-linearities are accounted for.

Hence, between iterations, there is no need to fully resolve pressure-velocity coupling. Which means between iterations, one does not need to actively ensure that they are solved simultaneously.

Step 1

Same as PISO, this is the momentum predictor step, we can predict a velocity field using the previous steps

$$\mathbf{u}_p^{predictor} = \frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p} + \frac{\nabla \cdot \tau/\rho}{a_p} + g/a_p \tag{1.18}$$

However, before going to step 2, we

Step 2

Also same as PISO, we do the pressure equation step

With this, we can formulate a new $\frac{\mathbf{H}(\mathbf{u}^t)}{a_p}$:

$$\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p} \tag{1.19}$$

With this, we can substitute into the mass balance (pressure equation):

$$\sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p}\right)_f = \sum_f \mathbf{S}_f \cdot \left(\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p}\right)_f$$

This allows for a new pressure field. Note that the gravity terms are missing because hydrostatic pressure is ignored.

$$\sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p}\right)_f$$

It is conceivable that this results in some small matrix where pressure fields are simultaneously solved.

This is the pressure solution within Jasak's thesis page 147 [Jasak, 1996].

Step 3

We now have a new pressure field based on mass balance:

$$\sum_f \mathbf{S}_f \cdot \left(\frac{P/\rho}{a_p}\right)_f$$

And a new predicted H field based on the initial predictor step:

$$\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p}$$

Now, the predictor H field is not guaranteed to allow continuity with the old velocity, but since it was used to generate a new pressure field using the continuity equation, it should be able to guarantee creation of a new velocity field which will satisfy continuity equation.

$$\mathbf{u}_p^{corrected} = \frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p} - \left(\frac{\nabla P/\rho}{a_p}\right)_{\text{corrected pressure field}} \tag{1.20}$$

repeated iterations

This forms the basis for iterating through both mass and momentum balances. This is one round of SIMPLE prediction-correction.

We may iterate this several times until the residual is small between consecutive iterations.

Now, it must be noted that previously during PISO, we do NOT repeat step 1 to recalculate as mentioned in page 148 Jasak's thesis Jasak, 1996:

$$\frac{\mathbf{H}(\mathbf{u}^{predictor})}{a_p}$$

Now, in SIMPLE, we do change H during subsequent iterations in the momentum predictor step [Jasak, 1996].

Dr Aiden's YouTube videos were helpful in understanding this regard too.

Relaxation Factors

However, between each iteration, timesteps may be large. Hence, the solution may experience some form of instability. Therefore, there is the introduction of relaxation factors which updates the velocity slowly between successive momentum predictor steps.

Jasak mentions the use of velocity under-relaxation factors α_u. as well as pressure under-relaxation factors α_p.

He outlined the following steps on page 148-149 of his thesis [Jasak, 1996]. For pressure:

$$p^{new} = p^{old} + \alpha_p(p^{predictor} - p^{old})$$

For momentum, a similar thing applies. Though for momentum α_u is usually around 0.8 and for pressure α_p is around 0.2. Jasak writes this in page 149 of his thesis [Jasak, 1996].

1.6 PIMPLE Algorithm

For the PIMPLE algorithm, this is a merger between PISO and SIMPLE algorithms [Greenshields and Weller, 2022].

Basically, every SIMPLE iteration now has the option to perform a set number of PISO iterations as well. That is perform the corrector steps multiple times per SIMPLE iteration. The PISO loops are known as inner loops, whereas the outer loops are known as SIMPLE loops.

It is then up to the user to know how many loops are required.

With this understanding, it is good to try and apply this thought process to our four control volume situation.

1.7 Application to Rankine Cycle

Mass, Energy and Momentum Balance Equations

Consider four control volumes. We consider the following PDEs:

$$\frac{\partial m}{\partial t} = \sum_i m_i \tag{1.21}$$

$$\frac{\partial(\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u}\mathbf{u}) = -\nabla P + \nabla \cdot \tau + \rho g \tag{1.22}$$

$\nabla \cdot \tau$ represents shear, ρg represents the gravity body force, $-\nabla P$ represents the pressure gradient.

$$V_j \frac{\partial \rho_i h_i}{\partial t} + \sum_j \dot{m}_j h_j = Q_i - W_i + \sum_j \tau_j \cdot \mathbf{u}_j \tag{1.23}$$

For a 1D case, we can consider Equations from Modelica. This was the excerpt from my dissertation [Ong, 2024]:

The mass balance equation for a 1D geometry is [Casella, Leva, et al., 2003].

$$A_{XS} \frac{\partial \rho}{\partial t} + \frac{\partial \dot{m}}{\partial x} = 0 \tag{1.24}$$

Where A_{XS} is the cross sectional area, ρ is the density of the differential volume dV, $\dot{m}$ is the mass flow rate, and x is the length coordinate in the direction of flow.

The momentum balance is [Casella, Leva, et al., 2003]:

$$\frac{\partial \dot{m}}{\partial t} + \frac{f_{fanning} P_w}{2\rho A_{XS}^2} \dot{m}|\dot{m}| + \rho g A_{XS} \frac{\partial z}{\partial x} + A_{XS} \frac{\partial P}{\partial x} = 0 \tag{1.25}$$

In Equation 1.25, in addition to the terms already defined for Equation 1.24, we have Fanning friction factor $f_{fanning}$, wetted perimeter P_w, acceleration due to gravity g, height for calculating hydrostatic pressure z and pressure P. The absolute mass flow rate $\dot{m}$ is used to account for flow reversal in the momentum equation. Additionally the derivative $\frac{\partial \dot{m}}{\partial t}$ is often set to zero to avoid fast pressure oscillations [Casella, Leva, et al., 2003]. The timescale of these oscillations is quite often connected to the speed of sound in that medium.

The energy balance is [Casella, Leva, et al., 2003]:

$$\rho A_{XS} \frac{\partial h_{enthalpy}}{\partial t} + A_{XS} \frac{\partial P}{\partial t} = -\rho A_{XS} u \frac{\partial h_{enthalpy}}{\partial x} + q_{wall} \frac{A_{wall}}{L} \tag{1.26}$$

In Equation 1.26, in addition to the previously defined terms for Equation 1.24 and 1.25, I define $h_{enthalpy}$ for specific enthalpy per unit mass for the control volume, average

fluid velocity u and q_{wall}, the heat flux received or lost through lateral surfaces. $h_{enthalpy}$ is denoted as such to prevent confusion with heat transfer coefficient, also commonly denoted h. $\frac{A_{wall}}{L}$ is the heated surface area of the wall A_{wall} per unit length, where L is the length of the 1D geometry. For tubes, $\frac{A_{wall}}{L} = P_w$ [Casella, Leva, et al., 2003].

If we consider the 1D balances:

$$A_{XS}\frac{\partial \rho}{\partial t} + \frac{\partial \dot{m}}{\partial x} = 0$$

$$\frac{f_{fanning}P_w}{2\rho A_{XS}^2}\dot{m}|\dot{m}| + \rho g A_{XS}\frac{\partial z}{\partial x} + A_{XS}\frac{\partial P}{\partial x} = 0$$

$$\rho A_{XS}\frac{\partial h_{enthalpy}}{\partial t} + A_{XS}\frac{\partial P}{\partial t} = -\rho A_{XS} u \frac{\partial h_{enthalpy}}{\partial x} + q_{wall}\frac{A_{wall}}{L}$$

Of course, for the equation of state, we can use a real gas equation or steam table. That is done in tampines-steam-tables.

rhoPimpleFoam

For the algorithm considered, we are going to translate from rhoPimpleFoam to what we want:

```
/*
    ------------------------------------------------------------------------*\
  ========= |
  \\ / F ield | OpenFOAM: The Open Source CFD Toolbox
   \\ / O peration |
    \\ / A nd | www.openfoam.com
     \\/ M anipulation |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2019 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
```

```
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM. If not, see <http://www.gnu.org/licenses/>.

Application
    rhoPimpleFoam

Group
    grpCompressibleSolvers

Description
    Transient solver for turbulent flow of compressible fluids for HVAC and
    similar applications, with optional mesh motion and mesh topology
       ↪ changes.

    Uses the flexible PIMPLE (PISO-SIMPLE) solution for time-resolved and
    pseudo-transient simulations.

Note
   The motion frequency of this solver can be influenced by the presence
   of "updateControl" and "updateInterval" in the dynamicMeshDict.

\*---------------------------------------------------------------------------
    ↪ */

#include "fvCFD.H"
#include "dynamicFvMesh.H"
#include "fluidThermo.H"
#include "turbulentFluidThermoModel.H"
#include "bound.H"
#include "pimpleControl.H"
#include "pressureControl.H"
#include "CorrectPhi.H"
#include "fvOptions.H"
#include "localEulerDdtScheme.H"
```

```
#include "fvcSmooth.H"

// * * *
```

```
        // Store divrhoU from the previous mesh so that it can be mapped
        // and used in correctPhi to ensure the corrected phi has the
        // same divergence
        autoPtr<volScalarField> divrhoU;
        if (correctPhi)
        {
            divrhoU.reset
            (
                new volScalarField
                (
                    "divrhoU",
                    fvc::div(fvc::absolute(phi, rho, U))
                )
            );
        }

        if (LTS)
        {
            #include "setRDeltaT.H"
        }
        else
        {
            #include "compressibleCourantNo.H"
            #include "setDeltaT.H"
        }

        ++runTime;

        Info<< "Time = " << runTime.timeName() << nl << endl;

        // --- Pressure-velocity PIMPLE corrector loop
        while (pimple.loop())
        {
            if (pimple.firstIter() || moveMeshOuterCorrectors)
            {
                // Store momentum to set rhoUf for introduced faces.
                autoPtr<volVectorField> rhoU;
                if (rhoUf.valid())
                {
                    rhoU.reset(new volVectorField("rhoU", rho*U));
                }

```

```
            // Do any mesh changes
            mesh.controlledUpdate();

            if (mesh.changing())
            {
                MRF.update();

                if (correctPhi)
                {
                    // Calculate absolute flux
                    // from the mapped surface velocity
                    phi = mesh.Sf() & rhoUf();

                    #include "correctPhi.H"

                    // Make the fluxes relative to the mesh-motion
                    fvc::makeRelative(phi, rho, U);
                }

                if (checkMeshCourantNo)
                {
                    #include "meshCourantNo.H"
                }
            }
        }

        if (pimple.firstIter() && !pimple.SIMPLErho())
        {
            #include "rhoEqn.H"
        }

        #include "UEqn.H"
        #include "EEqn.H"

        // --- Pressure corrector loop
        while (pimple.correct())
        {
            if (pimple.consistent())
            {
                #include "pcEqn.H"
            }
            else
```

```
                {
                    #include "pEqn.H"
                }
            }

            if (pimple.turbCorr())
            {
                turbulence->correct();
            }
        }

        rho = thermo.rho();

        runTime.write();

        runTime.printExecutionTime(Info);
    }

    Info<< "End\n" << endl;

    return 0;
}


// ************************************************************************
    ↪ //
```

Listing 1.1: rhoPimpleFoam.C

Discretisation

Considering the 1D control volume, and a 1D version of the divergence theorem, we can write:

$$\Delta x A_{XS} \frac{\partial \rho}{\partial t} + \Delta x \frac{\partial \dot{m}}{\partial x} = 0$$

$$V \frac{\partial \rho}{\partial t} + \sum_j \dot{m}_{j,face} = 0$$

As with before, we must estimate $\dot{m}_{j,face}$ using the centroid values. We can use upwinding interpolation, which would make the most sense.

That means, using p to denote the value at the centroid of the cell, and n to denote value at centroid of neighbouring cell.

At the face that is going to some adjacent cell,

If $\dot{m}_p \cdot \hat{S} > 0$:

$$\dot{m}_{j,face} = \dot{m}_p$$

else

$$\dot{m}_{j,face} = \dot{m}_n$$

$\hat{S}$ is the unit area surface normal vector that points away from the centroid of the cell.

Step 1: Density Equation

For temporal time stepping:

$$V\frac{\rho^{t+\Delta t} - \rho^t}{\Delta t} + \sum_j \dot{m}_{j,face} = 0$$

We have an explicit equation that allows us to determine a guess in density for the next iteration if we were to do it explicitly.

This is the first step in the rhoPimpleFoam algorithm. (see line 169)

If there are any flows in or out of here, we can incorporate it as well into this equation.

Of course, $\rho^{t+\Delta t}$ is not exactly the correct term, as we haven't completely solved and iterated the new density fields. $\rho^{predicted}$ should be the correct term. So what we have now is an intermediate value:

$$V\frac{\rho^{predicted} - \rho^t}{\Delta t} + \sum_j \dot{m}_{j,face} = 0$$

Step 2: Momentum Predictor Step

For the momentum predictor, we start with:

$$\frac{f_{fanning}P_w}{2\rho A_{XS}^2}\dot{m}|\dot{m}| + \rho g A_{XS}\frac{\partial z}{\partial x} + A_{XS}\frac{\partial P}{\partial x} = 0$$

Now, we know that the densities have been predicted based on mass balance, we can do a momentum predictor step. Note that we ignore the $\frac{\partial \dot{m}}{\partial t}$ term just like modelica. This makes it easier for us. The only thing for us to discrete here is the pressure equation. We will use divergence theorem again by integrating each term by volume.

However, the 1D form has already been integrated across the areas, so we only need do is over dx:

$$\Delta x \frac{f_{fanning} P_w}{2\rho A_{XS}^2} \dot{m}|\dot{m}| + \rho g A_{XS} \int dx \frac{\partial z}{\partial x} + A_{XS} \int dx \frac{\partial P}{\partial x} = 0$$

$$\Delta x \frac{f_{fanning} P_w}{2\rho A_{XS}^2} \dot{m}|\dot{m}| + \rho g A_{XS} \Delta z + A_{XS} \Delta P_{x,face} = 0$$

For this, we can do some pressure interpolation using some weighting factor α:

$$\Delta P_{x,face} = \alpha P_p + (1-\alpha) P_n$$

OpenFOAM tends to correlate α based on the distance from the cell centroid to the boundary. In the 1D case, α is simply based on the ratios of the lengths of each 1D cell.

For this, we can see

$$1 - \alpha = \frac{\text{cell P length}}{\text{cell P length} + \text{cell N length}}$$

Hence,

$$\alpha = \frac{\text{cell N length}}{\text{cell P length} + \text{cell N length}}$$

Now, the density would be based on the previous density predictor step, but the pressures and other properties would be based on the previous timestep.

$$\Delta x \frac{f_{fanning} P_w}{2\rho A_{XS}^2} \dot{m}^{predicted}|\dot{m}^{predicted}| + \rho g A_{XS} \Delta z + A_{XS} \Delta P_{x,face} = 0$$

As for whether we should take the positive or negative answer of $|\dot{m}^{predicted}|$, that should depend on the sign of the total pressure change.

Now, it is evident that this is a nonlinear equation. So we want to linearise this equation just like for PIMPLE. We shall use the mass flowrates from the previous timesteps for the fanning friction factor and $\dot{m}$.

$$\Delta x \frac{f_{fanning}^t P_w}{2\rho^{predicted} A_{XS}^2} \dot{m}^t|\dot{m}^{predicted}| + \rho^{predicted} g A_{XS} \Delta z + A_{XS} \Delta P_{x,face}^t = 0$$

Now, solving for this is kind of cumbersome due to the presence of the modulus sign, it may be better to do:

$$\Delta x \frac{f_{fanning}^t P_w}{2\rho^{predicted} A_{XS}^2} \dot{m}^{predicted}|\dot{m}^t| + \rho^{predicted} g A_{XS} \Delta z + A_{XS} \Delta P_{x,face}^t = 0$$

Of course, if area and such were changing, we could use this from the last timestep. Nevertheless, this is a control volume, and we should not expect area to change.

This makes it relatively easy to obtain $\dot{m}^{predicted}$ as the steps are quite explicit.

Step 3: Energy Equation

With the predicted mass flowrates, and densities, we can then consider the energy equation.

$$\rho A_{XS}\frac{\partial h_{enthalpy}}{\partial t} + A_{XS}\frac{\partial P}{\partial t} = -\rho A_{XS} u \frac{\partial h_{enthalpy}}{\partial x} + q_{wall}\frac{A_{wall}}{L}$$

$$\rho A_{XS}\frac{\partial h_{enthalpy}}{\partial t} + A_{XS}\frac{\partial P}{\partial t} = -\dot{m}\frac{\partial h_{enthalpy}}{\partial x} + q_{wall}\frac{A_{wall}}{L}$$

Let us integrate across dx:

$$\rho V\frac{\partial h_{enthalpy}}{\partial t} + V\frac{\partial P}{\partial t} = -\sum_j \dot{m}_j h_j + q_{wall}A_{wall}$$

Now, this is tricky because we have two time derivatives. One based on pressure, one based on enthalpy. What do we do with this?

Based on the OpenFOAM EEqn.H in rhoPimpleFoam:

```
{
    volScalarField& he = thermo.he();

    fvScalarMatrix EEqn
    (
        fvm::ddt(rho, he) + fvm::div(phi, he)
      + fvc::ddt(rho, K) + fvc::div(phi, K)
      + (
            he.name() == "e"
          ? fvc::div
            (
                fvc::absolute(phi/fvc::interpolate(rho), U),
                p,
                "div(phiv,p)"
            )
          : -dpdt
        )
      - fvm::laplacian(turbulence->alphaEff(), he)
     ==
        fvOptions(rho, he)
    );

    if (MRF.active())
    {
        EEqn += fvc::div(MRF.phi(), p);
    }
```

```

    EEqn.relax();

    fvOptions.constrain(EEqn);

    EEqn.solve();

    fvOptions.correct(he);

    thermo.correct();
}
```

It appears we have to calculate a pressure gradient first. Not sure where we get that from though!

Now, for the **first** iteration, dpdt is zero, because there is no prior pressure from which to calculate it.

In subsequent steps though, p will be updated. And one can have a dpdt as

$$\frac{\partial p}{\partial t} = \frac{p^t - p^{t-\Delta t}}{\Delta t}$$

Equation of State:

Now, with this, we will then be able to solve for enthalpy at the next timestep.

Now, we have predicted a new density ρ, a new set of mass flowrates $\dot{m}$ and a new enthalpy h. With density and enthalpy, in theory, one should be able to obtain a new pressure field.

$$p = p(\rho, h)$$

For a ideal gas equation of state, this is quite easy if the heat capacity equation is straightforward, as pressure is explicitly in terms of specific volume $1/\rho$ and temperature.

However, for cubic equation of state, then we need to find temperature iteratively from h.

And for steam tables, we need to iteratively find the temperature which gives us the correct density ρ using the p, h equations.

Hence, for steam tables, we need to vary p such that:

$$\rho(p, h) - \rho_{set} = 0$$

By doing so, we will obtain our pressure and (p,h) point.

Now, this step is not done in the EEqn.H step, the algorithm calls for finding the temperature first from the thermodynamic states of ρ and h so that we will get a ρ, T point. This is ideal for putting into a cubic equation of state to find p.

Step 4

Now, we enter the PISO part, this is the pressure corrector step. Essentially, it is the momentum equation substituted into the pressure equation.

In the pEqn.H, we see pimpleFoam accounts for transonic equations, but we shall ignore that. We get:

```
if (!pimple.SIMPLErho())
{
    rho = thermo.rho();
}

// Thermodynamic density needs to be updated by psi*d(p) after the
// pressure solution
const volScalarField psip0(psi*p);

volScalarField rAU(1.0/UEqn.A());
surfaceScalarField rhorAUf("rhorAUf", fvc::interpolate(rho*rAU));
volVectorField HbyA(constrainHbyA(rAU*UEqn.H(), U, p));

if (pimple.nCorrPISO() <= 1)
{
    tUEqn.clear();
}

surfaceScalarField phiHbyA
(
    "phiHbyA",
    fvc::interpolate(rho)*fvc::flux(HbyA)
  + MRF.zeroFilter(rhorAUf*fvc::ddtCorr(rho, U, phi, rhoUf))
);

fvc::makeRelative(phiHbyA, rho, U);
MRF.makeRelative(fvc::interpolate(rho), phiHbyA);

// Update the pressure BCs to ensure flux consistency
constrainPressure(p, rho, U, phiHbyA, rhorAUf, MRF);

// NOTE: I'm skipping the transonic part

{
    fvScalarMatrix pDDtEqn
    (
```

```
        fvc::ddt(rho) + psi*correction(fvm::ddt(p))
      + fvc::div(phiHbyA)
     ==
        fvOptions(psi, p, rho.name())
    );

    while (pimple.correctNonOrthogonal())
    {
        fvScalarMatrix pEqn(pDDtEqn - fvm::laplacian(rhorAUf, p));

        pEqn.solve(p.select(pimple.finalInnerIter()));

        if (pimple.finalNonOrthogonalIter())
        {
            phi = phiHbyA + pEqn.flux();
        }
    }
}


// Explicitly relax pressure for momentum corrector
p.relax();

U = HbyA - rAU*fvc::grad(p);
U.correctBoundaryConditions();
fvOptions.correct(U);
K = 0.5*magSqr(U);

if (pressureControl.limit(p))
{
    p.correctBoundaryConditions();
}

thermo.correctRho(psi*p - psip0, rhoMin, rhoMax) ;

#include "rhoEqn.H"
#include "compressibleContinuityErrs.H"

rho = thermo.rho();

// Correct rhoUf if the mesh is moving
fvc::correctRhoUf(rhoUf, rho, U, phi);
```

```

if (thermo.dpdt())
{
    dpdt = fvc::ddt(p);

    if (mesh.moving())
    {
        dpdt -= fvc::div(fvc::meshPhi(rho, U), p);
    }
}
```

We recall that the corrector step was done by substituting the expressions in for momentum balance into the mass conservation equations.

So we have:

$$V\frac{\rho^{t+\Delta t}-\rho^t}{\Delta t}+\sum_j \dot{m}_{j,face}=0$$

$$\frac{f_{fanning}P_w}{2\rho A_{XS}^2}\dot{m}|\dot{m}|+\rho g A_{XS}\frac{\partial z}{\partial x}+A_{XS}\frac{\partial P}{\partial x}=0$$

Now, we just had intermediate guesses from

- density equation ρ
- the momentum predictor, predicting $\dot{m}$
- energy equation, predicting h

The next few steps use the pressure-velocity coupling steps from the SIMPLE and PISO algorithms, but there is a need to update it for density.

Recall for **incompressible** that the momentum equation can be written as:

$$\mathbf{u}_p^{predictor}=\frac{\mathbf{H}(\mathbf{u}^t)}{a_p}-\frac{\nabla P/\rho}{a_p}+\frac{\nabla\cdot\tau/\rho}{a_p}+g/a_p$$

The pressure equation is done by substituing this equation into the continutity equation:

$$\nabla\cdot\mathbf{u}=0$$
$$\nabla\cdot\left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p}-\frac{\nabla P/\rho}{a_p}+\frac{\nabla\cdot\tau/\rho}{a_p}+g/a_p\right)=0$$

By doing so, we can have guesses for the pressure in terms of momentum, or velocity guesses:

$$\nabla\cdot\frac{\nabla P/\rho}{a_p}=\nabla\cdot\left(\frac{\mathbf{H}(\mathbf{u}^t)}{a_p}+\frac{\nabla\cdot\tau/\rho}{a_p}+g/a_p\right)$$

This was the original equation mentioned in Jasak's thesis [Jasak, 1996].

That is well and good, but now we have another layer of complication. We move from a pressure based solver to a density based solver. Bone outlines this thought process well [Bone, 2020]. Bone then cites Kurganov et al. for coming up with the central schemes (rhoCentralFoam) [Kurganov and Tadmor, 2000].

Bone mentiones rhoSimpleFoam on page 6-7 [Bone, 2020], this is essential for understanding rhoPimpleFoam. Now, in page 3 of the thesis, Bone mentions about steady state compressible equations.

In that regard, continutity equation is similar to SIMPLE algorithms for incompressible flow, except that ρ is added in. Time derivative terms are not accounted for.

For page 7 onwards, Bone writes about the PISO algorithm for compressible flow [Bone, 2020]. This is more relevant to our case. However, Bone does not elaborate on how density is solved for in rhoPimpleFoam. This is also not covered in the rhoCentralFoam part of the thesis. However, the solution procedure is standard, density, momentum, energy, then EOS [Bone, 2020].

Clues from sonicFoam can also be obtained from Canteros [Canteros and Polanskỳ, 2024] as sonicFoam uses the PIMPLE algorithm like rhoPimpleFoam.

Now, anyway, the first thing that is done is to use the Equation of state to find a pressure.

So now, we have these values which are guessed from before:

- From density eqn: $\rho^{predicted}$
- From momentum predictor: $\dot{m}^{predicted}$
- From energy equation: $h^{predicted}$
- From Equation of State: ρ, T from ρ, h

Now we have:

- p from ρ, T (or ρ, h, should not be an issue either)

So far, there is minimal coupling between momentum and mass balance equation. Hence, we are using these two equations to form the pressure equation from before:

Note that after the pressure equation is solved, the mass conservation equation is solved again, and then the density is updated using the equation of state.

This is repeated many times during the inner PISO loop.

It is also here that dpdt is updated the end of each pEqn.H step.

First let's rearrange to things in terms of H and a. And we will do this calculation in terms of mass flowrate rather than u for convenience.

Now, all the viscous and other terms (other than pressure, can be subsumed in H).

$$\mathbf{u}_p^{predictor} = \frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p}$$

$$a_p = \frac{f_{fanning} P_w}{2\rho A_{XS}^2} |\dot{m}| \Delta x$$

Though for our adaptation, there is no $\frac{\partial m}{\partial t}$ term. We need to make simplifications as there are no more time dependent terms.

We note that H is the non pressure term (we ignore time derivative in this case as per Modelica simplifications):

$$H = \rho g A_{XS} \Delta z$$

So the H/A term is:

$$H/A = \frac{\rho g A_{XS} \Delta z}{\frac{f_{fanning} P_w}{2\rho A_{XS}^2} |\dot{m}| \Delta x}$$

Bibliography

Devolder, Brecht et al. (2015). "A review of the implicit motion solver algorithm in OpenFOAM® to simulate a heaving buoy". In: *18th Numerical Towing Tank Symposium (NuTTS'15)*.

Penttinen, Olle, Ehsan Yasari, and Håkan Nilsson (2011). "A pimplefoam tutorial for channel flow, with respect to different LES models". In: *Practice Periodical on Structural Design and Construction* 23.2, pp. 1–23.

Jasak, Hrvoje (1996). "Error analysis and estimation in the Finite Volume method with applications to fluid flows". In.

Jasak, Hrvoje, Aleksandar Jemcov, Zeljko Tukovic, et al. (2007). "OpenFOAM: A C++ library for complex physics simulations". In: *International workshop on coupled methods in numerical dynamics*. Vol. 1000. Dubrovnik, Croatia), pp. 1–20.

Greenshields, Christopher J and Henry G Weller (2022). "Notes on computational fluid dynamics: General principles". In: *(No Title)*.

Ong, Theodore Kay Chen (2024). *Digital Twins as Testbeds for Iterative Simulated Neutronics Feedback Controller Development*. University of California, Berkeley.

Casella, Francesco, Alberto Leva, et al. (2003). "Modelica open library for power plant simulation: design and experimental validation". In: *Proceeding of the 2003 Modelica conference, Linkoping, Sweden*. Citeseer.

Bone, Sean (2020). "Comparative study of density-based versus pressure-based solvers for supersonic flow". In: *Eidgenössische Technische Hochschule Zürich Swiss Federal Institute of Technology Zurich*.

Kurganov, Alexander and Eitan Tadmor (2000). "New high-resolution central schemes for nonlinear conservation laws and convection–diffusion equations". In: *Journal of computational physics* 160.1, pp. 241–282.

Canteros, Maria Laura and Jiří Polanskỳ (2024). "Review and comparison of two OpenFOAM® solvers: rhoCentralFoam and sonicFoam". In: *EPJ Web of Conferences*. Vol. 299. EDP Sciences, p. 01005.